Lecture 11: Floating Point Numbers

Leili Rafiee Sevyeri

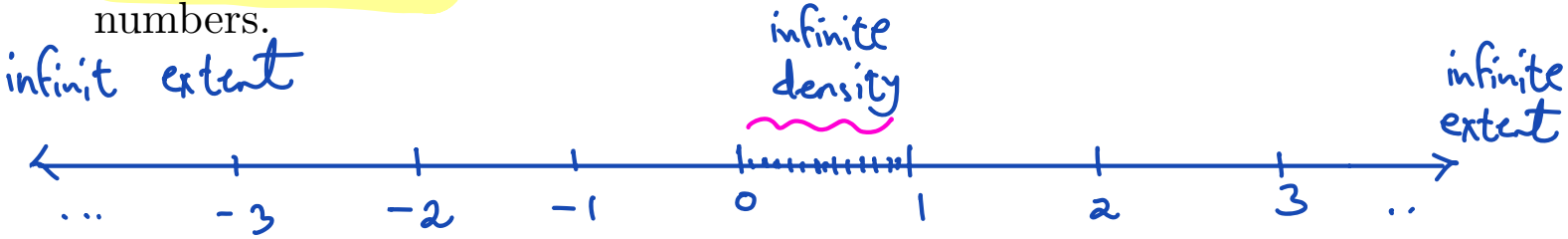
Based on lecture notes by me and many previous CS335 instructors

Fall 2026
Cheriton School of Computer Science
University of Waterloo

Real Numbers $\mathbb{R}$

For any number x_1, there exists another number x_2, s.t, $|x_1| < |x_2|$.

- Infinite in extent: There exists x such that $|x|$ is arbitrarily large.
- Infinite in *density*: Any interval $a \leq x \leq b$ contains infinitely many numbers.



Problem: Computers cannot represent infinite/arbitrarily large quantities!

Solution: The standard (partial) solution is to use floating point numbers to approximate real numbers.

Floating Point Systems

An approximate representation of real numbers using a finite number of bits.

Questions to ponder:

- How can we represent real numbers digitally?
- How does the resulting "approximate" number system behave?
- Why is this important?

Usage of the word “numerical”?

In ordinary language, it means: “having to do with numbers”.
For scientific/numerical computing, it instead means more like:

“having to do with a computer approximation of some mathematical value/problem, usually involving (approximate) real numbers”.

Note:

- An “analytical” solution is the exact answer to a problem.
- A “numerical” solution is the approximate one produced by our algorithm (often in floating point).

Numerical Errors: Toy Example

Consider the sum:

$$12 + \sum_{i=1}^{100} 0.01 = 12 + 100(0.01) = 13$$

True answer: 13.

Now, assume we can only keep 2 digits of accuracy at a time. What happens? Perform the sum one addition at a time, round after each.

$12.01 \approx 12$

$(((12 + 0.01) + 0.01) + 0.01) + \cdots \approx 12$

$12 + 0.01 = 12.01$ round it → keep 2 digits 12

"Numerical" answer: ?? 12

Numerical Errors: Taylor series example

Say we want to approximately evaluate $e^{-5.5}$ (without a handy exponential function).

Recall the Taylor series for a function :

$$f(x) = f(a) + (x-a)f'(a) + \frac{(x-a)^2}{2!}f''(a) + \frac{(x-a)^3}{3!}f^{(3)}(a) + \cdots$$

(Taylor series lets us approximate a function value in a neighbourhood, assuming we know its value and derivatives at some nearby point a.)

Numerical Errors: Taylor series example

Applying Taylor series to $f(x) = e^x$ around $a = 0$, gives

$$e^x \approx e^0 + (x-0)e^0 + \frac{(x-0)^2}{2!}e^0 + \frac{(x-0)^3}{3!}e^0 + \cdots = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots$$

Plugging in $x = -5.5$, and using 5 digits of accuracy, we find:

$$e^{-5.5} \approx 0.0026363 \qquad \text{(Why?)}$$

$e^{-5.5} \approx 1 - 5.5 + 15.125 - 27.730 + \cdots$

$= 0.0026363$

Trying it on your calculator/phone:

$e^{-5.5} = 0.0040868$ (rounded to 5 digits).

Numerical Errors: Taylor series example

Instead of working with $e^{-5.5}$, we work with $(e^{5.5})^{-1}$, because $e^{-x} = (e^x)^{-1}$. So using Taylor series:

Another approximation is given by: $e^{-x} = \frac{1}{e^x} = \frac{1}{1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots}$

Again using 5 digits accuracy, we now get: $e^{-5.5} \approx 0.0040865$

This is *much* closer to the desired solution, $e^{-5.5} = 0.0040868$

Course notes contain a 2nd example of a numerical approximation going badly astray.

What do we learn? All of the terms in the second method are positive, so there can be no cancellation of information.

What went wrong here?

Floating point numbers often don't quite behave like true real numbers.

This can lead to subtle (yet huge) errors!

To be useful, our numerical algorithms must work effectively under *floating point arithmetic*.

Let's examine the floating point representation.

Infinite Expansion of Real Numbers

We can express a real number as an *infinite expansion* relative to some *base* β.

e.g., consider $\frac{73}{3} \approx 24.33333\cdots$

In base 10:

$$\frac{73}{3} = 24.3333\cdots = 2\times 10^{1} + 4\times 10^{0} + 3\times 10^{-1} + 3\times 10^{-2} + \cdots$$

In base 2:

$$\frac{73}{3} = 11000.010101\ldots = 1\times 2^{4} + 1\times 2^{3} + 0\times 2^{2} + 0\times 2^{1} + 0\times 2^{0} + \cdots$$

Normalized Form of a Real number

After expressing the real number in the desired base β, we multiply by a power of β to shift it into a **normalized form**:

$$0.d_1 d_2 d_3 d_4 \cdots \times \beta^p \qquad (d_1 \neq 0)$$

where...

- d_i are digits in base β, i.e., $0 \leq d_i < \beta$
- *normalized* implied we shift to ensure $d_1 \neq 0$
- exponent p is an integer

For example, $24.3333\ldots = 0.243333\ldots \times 10^2$

How can we now avoid storing infinite data?

Careful: You may see other normalization conventions outside this class, e.g. $d_1.d_2 d_3 d_4 \cdots \times \beta^p$ ⇝ not in this course ☺

Floating Point

Density (or *precision*) is bounded by limiting the number of digits, t.

Extent (or *range*) is bounded by limiting the range of values for exponent p.

Our floating point representation then has the finite form:

$$\pm 0.d_1 d_2 \cdots d_t \times \beta^p$$

for $L \leq p \leq U$, and $d_1 \neq 0$.

or

0 as a special case.

The four integer parameters $F = \{\beta, t, L, U\}$ characterize a specific floating point system, F, where β = base, t = mantissa, and L and U are the lower and upper bounds for the exponent.

Overflow/underflow errors

- If the exponent is too big ($> U$) or too small ($< L$), our system cannot represent the number.
- When arithmetic operations generate such a number, this is called overflow (too big) or underflow (too small), respectively.
- For underflow, answer simply rounds to 0 instead.
- For overflow, will typically produce a $\pm\infty$ (or NaN - not a number).

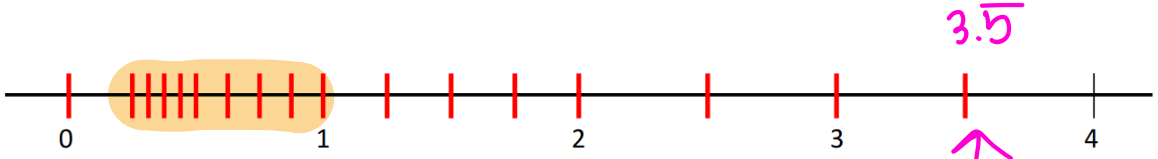
Exception handling:

	example	result
Invalid op.	$\frac{0}{0}$	NaN
Division by 0	$\frac{1}{0}$	$\pm\infty$
Overflow	$N_{max} + 1$	$\pm\infty$
Underflow	$N_{min}/2$	0

Floating Point Density

Fixed point numbers: The number of digits after the radix (decimal) point is fixed. For ex. 3 digits after decimal: 10.234 and 171.001

Unlike fixed point, floating point numbers are not evenly spaced! E.g., For $F = \{\beta = 2, t = 3, L = -1, U = 2\}$ the representable (non-negative) values are spaced like:



$$0, 0.100 \times 2^{-1}, 0.101 \times 2^{-1}, 0.110 \times 2^{-1}, 0.111 \times 2^{-1}, 0.100 \times 2^{0},$$
$$0.101 \times 2^{0}, 0.110 \times 2^{0}, \ldots\ldots\ldots, 0.110 \times 2^{2}, 0.111 \times 2^{2}$$

3.5

0.100×2^{-1}: $(0.100)_2 = 1 \times 2^{-1} + 0 \times 2^{-2} + 0 \times 2^{-3}$

$= 0.5$

$0.5 \times 2^{-1} = 1/4$

Rounding vs. Truncation ("Chopping")

Floating point systems offer different *rounding modes* when converting a real number into a representable FP number.

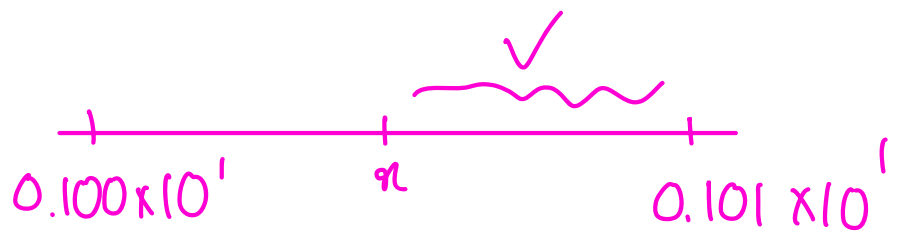
We will only consider:

- **Round-to-nearest** - rounds to closest available number in F.
  - Usually the default.
  - We'll break ties by simply rounding $1/2$ up
    (various other options exist...)
- **Truncation/"Chopping"** - rounds to the next number in F *towards zero*.
  - i.e. Simply discard any digits after the t-th digit.

For ex. $F = \{ \beta = 10,\ t = 3,\ L = -5,\ U = 5 \}$, and

$x = 0.1005 \times 10^{1}$ ⟶ closest available number to x in F :

- ⟶ 0.100×10^{1}
- 0.101×10^{1} ✓



Example 1

Express 253.9 in a floating point system with base $\beta = 10, t = 6$ digits, $L = -5, U = 5$.

Process:

1. Express in base $\beta = 10$: 253.9
2. Shift to get leading 0, set exponent p: $0.25\,39 \times 10^{3} \Longrightarrow p = 3$
3. Pad or round/truncate to t digits: 0.253900×10^{3}

Example 2

Express 8.25 in a floating point system with base $\beta = 2$, $t = 7$ digits, $L = -5, U = 5$.

Process:

1. Express in base $\beta = 2$: 1000.01

$8 = 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 1000$

$0.25 = \frac{1}{4} = 0 \times 2^{-1} + 1 \times 2^{-2} = 0.01$

2. Shift to get leading 0, set exponent p: 0.100001×2^4, $p = 4$
3. Pad or round/truncate to t digits: 0.1000010×2^4

Example 3

Express 1030.9671 in a floating point system with base $\beta = 10$, $t = 6$ digits, $L = -5, U = 5$

Process:

1. Express in base $\beta = 10$: 1030.9671
2. Shift to get leading 0, set exponent p: 0.10309671×10^{4}, $p=4$
3. Pad or round/truncate to t digits:
   - With rounding: 0.103097×10^{4}
   - With truncation: 0.103096×10^{4}

Measuring Error

If $x_{exact} = 10$, then the ratio is $\frac{10}{10} = 1 = 100\%$ error.

Our algorithms will compute approximate solutions to problems.

Difference between ...

x_{exact} = true solution, and

x_{approx} = approximate solution,

... gives the error of an algorithm.

If the absolute value of this difference is 10, is this a big error?

We distinguish between absolute error and relative error.

If $x_{exact} = 10^6$, then the ratio of the error to the value is $\frac{10}{10^6} = 0.00001 \rightarrow$ small.

End of Lecture 11

Absolute vs. Relative Error

Absolute error:

$$E_{abs} = |x_{exact} - x_{approx}|$$

Relative error:

$$E_{rel} = \frac{|x_{exact} - x_{approx}|}{|x_{exact}|}$$

Example

Let $x_{exact} = 220$, $x_{approx} = 198$

Compute E_{abs} and E_{rel}.

What does each tell us?

Note: sometimes we'll also consider the *signed* error, i.e. without abs. values here.